

CSCE 410 P1 Report: Logging Functionality

Zayd Nadir

1. Submission information

a. Repository URL: <https://github.com/zaydn4321/333006458-xv6-csce410-p1>

b. Late-day tokens: No, none used.

2. Resources used

a. Which resources did you leverage while developing your solution?

I used Claude Code along with the assignment handout and the existing xv6 source code. I wrote the main parts of the code myself, including the `dprintf` macro, the `debugctl` system call, the `dbg` tool, and the instrumentation in `proc.c` and `exec.c`. Some parts were generated by Claude Code.

b. How were they useful?

I used Claude Code to clarify parts of the assignment and instructions I wasn't sure about before starting. When I had it generate code, I told it exactly what I wanted and it wrote it the way I asked. I also used it for syntax (C macros, GCC attributes, the `__atomic` builtins, Makefile changes) and for debugging when something didn't compile or behaved differently than I expected. The simulations for the demonstration were done with the help of Claude Code. It ran the test scenarios in xv6 and saved the transcripts. This report was also written with the help of Claude Code. I reviewed and tested everything it produced, and it was a tool I directed rather than the main source of the work.

c. What is your estimate for how much time these resources saved you?

About 8 to 10 hours. Most of that came from not having to search documentation and forums for syntax details and compiler errors, and from getting unstuck faster while debugging.

d. If you had to redo the assignment without any of the resources, how many hours do you think it would take you?

About 20 to 25 hours. The design wouldn't change, but I would have spent much longer looking up syntax, reading documentation on things like GCC atomics and xv6's build system, and tracking down bugs on my own. Writing the report and putting together the transcripts would also have taken longer.

3. Source code added or modified

New files:

File	What it has
kernel/debug.h	Categories, levels, debugctl op codes, the name tables, and the dprintf macro. The kernel and user/dbg.c both include it.
kernel/debug.c	dbg_mask, dbg_level, dbg_pid, the prefix helpers, and sys_debugctl.
user/dbg.c	The dbg user program.

Modified files:

File	Change
Makefile	Added debug.o to OBJS and _dbg to UPROGS. Added a DBG=0 build option.
kernel/syscall.h	Added SYS_debugctl 23.
kernel/syscall.c	Added the extern and the table entry for sys_debugctl.
user/usys.pl	Added entry("debugctl").
user/user.h	Added uint64 debugctl(int, uint64);
kernel/proc.c	Added messages in allocproc, freeproc, userinit, kfork, kexit, kwait, kkill and scheduler. reparent() now returns the number of children it moved.
kernel/exec.c	Added messages in kexec for success and failure.

The transcripts are in transcripts/. See section 4.

4. Transcripts (console output) included

All the files are in transcripts/. Each file is one full session from boot to shutdown. It has every command typed at the xv6 prompt and the output. Each session starts with a new fs.img.

File	What it shows
1_boot_silent.txt	Scenario 1. The mask is 0 at boot. dbg shows mask 0. echo hi and ls print no debug output.
2_proc_ls_echo.txt	Scenario 2. dbg on proc, then ls and echo hi. Each command shows kfork, kexec, kexit and kwait. The parent is the shell (pid 2) and the child pids match.
3_forktest.txt	Scenario 3. dbg on proc at level trace, then forktest. It shows allocproc and freeproc for every process, plus fork, exit and wait.

File	What it shows
3_forktest_check.txt	A check of 3_forktest.txt using grep and comm. The commands are in the file. Every forked pid exited, was reaped, and was freed once. There are no leaks and no double frees. The file also explains why the counts are off by one or two.
4_off_all.txt	Scenario 4. ls, echo hi and forktest with proc on give 4, 4 and 189 debug lines. After dbg off all, the same three commands give 0.
5_sched.txt	Scenario 5. dbg on sched by itself prints nothing. The sched message is at trace and the default level is info. After dbg level trace, echo hi gives 3 sched lines and forktest gives 106.
6_usertests.txt	Scenario 6. usertests with the mask at 0. ALL TESTS PASSED and there are no debug lines. The usertrap(): unexpected scause lines come from tests that fault on purpose.
7_bonus_pid_filter.txt	The bonus. See section 7.
8_extra_errors_levels.txt	Extra tests. Killing a pid that doesn't exist (WARN). Running a program that doesn't exist (ERR). The level set to warn and then err. Bad input to dbg.

transcripts/README.txt has the same list.

5. Design comments

I used the approach from parts A to D. These notes cover the parts that aren't obvious from the handout.

Part A: dprintf

A message prints only if three things are true. Its category bit is set in dbg_mask. Its level is at or below dbg_level. For the bonus, the pid filter also has to match. Every message starts with the category, the current pid and the function name:

```
[proc  ][2][kfork] parent 2 forked child 5, copied 20480 bytes
```

The category names are padded by hand in the table. Kernel printk doesn't support width like %-7s. grep '\[proc' log.txt gives only the process messages. The pid is -1 when no process is running on that CPU. This happens in userinit(), which runs from main() before the scheduler starts.

dprintf is a macro, not a function. It puts printk inside an if. So the arguments are only evaluated when the message prints. It also lets __func__ give the caller's name.

Compiling it out: `make DBG=0` defines `DBG_DISABLE`. Then `dprintf` becomes `if (0) printk(...)`. I didn't use an empty statement. With an empty statement, a variable that is only used in a log message would be unused. That is a warning, and `-Wall -Werror` would stop the build. With `if (0)` the compiler still sees the arguments, and `-O` removes the call. I checked this three ways:

- I disassembled one `dprintf` call. With logging on, it loads `dbg_mask`, does an `andi`, and branches. The arguments and the `printk` call are only on the taken branch. With `DBG=0`, the function is just `ret`.
- The whole kernel builds with `make DBG=0` with no warnings. Kernel text goes from 36,136 to 31,096 bytes.
- `strings kernel/kernel | grep -c 'reaped child'` gives 1 in the normal build and 0 with `DBG=0`. The messages are gone from the binary.

Name table: `dbg_cats[]` in `debug.h` has each bit, the name the user types, and the padded name the kernel prints. The kernel uses it for the prefix. `dbg` uses it to read names and to print its usage. So a new category only needs to be added in one place. The tables are `static` in a header, so they have `__attribute__((unused))`. Without it, every file that includes `debug.h` and doesn't use them would fail with `-Werror`.

Part B: debugctl

I kept the suggested interface. It is one system call, `uint64 debugctl(int op, uint64 arg)`, with `GETMASK`, `SETMASK`, `GETLEVEL` and `SETLEVEL`. The `SET` ops return the old value. A bad op or a level out of range returns -1. Because it is one syscall with op codes, the bonus only needed two new op codes and no new syscall. The level check is `arg > DBG_LEVEL_MAX`. `arg` is unsigned, so this also rejects negative numbers.

There is one problem with this interface. -1 as a `uint64` is `0xffffffffffffffff`, which is also `DBG_ALL`. So `GETMASK` with every category on looks the same as an error. I kept the interface from the handout. A separate error value or a pointer argument would fix it.

Part C: dbg

`dbg`, `dbg on`, `dbg off`, `dbg only`, `dbg level` and `dbg off all` work like the handout shows. `on`, `off` and `only` read the current mask, change it with `|=`, `&=` `~` or `=`, and write it back. `all` is `DBG_ALL`. Bad input gets an error message and exit status 1. That includes an unknown category or level, a missing argument, and an unknown command. The kernel checks the values again.

Part D: levels

Level	Messages
ERR	allocproc failures (no free slot, trapframe or pagetable), kfork failures, exec of a missing file, exec failure
WARN	kill of a pid that doesn't exist

Level	Messages
INFO	userinit, fork (parent, child, size), exec (path, argc, size), exit (pid, status, children reparented), wait (parent, child, status), kill (killer, target)
TRACE	allocproc success (pid, slot), freeproc (pid, name, slot), the scheduler message

At INFO you see what the system is doing. You see who forked who, what ran, who exited and who reaped it. TRACE adds the slot allocations and frees. That is only needed when looking for a leak. `8_extra_errors_levels.txt` shows the levels working at warn and err.

Other details:

- `freeproc()` prints before it clears `p->pid`.
- `reparent()` didn't return anything before. Now it returns how many children it moved, and `kexit` prints that. It is only called in `proc.c`.
- The `allocproc` success message prints while `p->lock` is held. This is allowed because the console lock is a leaf and `printk` doesn't sleep. `usertests` passes with the messages in.
- The scheduler message is after `c->proc = p`. It prints `p->pid` from the loop variable.

Making the scheduler message usable (scenario 5)

`5_sched.txt` shows the problem. One `forktest` run gave 106 sched lines and only 2 lines of real output. There is one line for every context switch, across 3 CPUs and about 60 processes. When the system is idle it's quiet. `xv6` puts idle CPUs in `wfi`, so the amount of output depends on the work being done. With more processes, or a long program that gets preempted every timer tick, it would never stop. To make it usable I would:

1. Use it with the pid filter from the bonus. Then you only see the switches for one process.
2. Count switches per CPU and per process instead of printing each one. Print a summary when asked, for example with a new `debugctl` op. Printing to the UART inside the scheduler is slow and changes the timing you are trying to look at.
3. Only print every Nth switch, or a few lines per timer tick.
4. Save the events in a ring buffer in memory and print them later.

Other notes

- The repo's GitHub workflow runs `make fmt` and fails if `clang-format` changes anything. The new code passes `clang-format`. The category table is inside `// clang-format off` and `// clang-format on`, the same as the table in `syscall.c`.
- The linker prints LOAD segment with RWX permissions for `kernel/kernel` and `user/_forktest`. The original tree prints both of these too with this version of `binutils`. I checked before making any changes. There are no compiler warnings. A fresh clone of the repo builds with no compiler warnings, both normally and with `make DBG=0`.

6. Concurrency

`dbg_mask`, `dbg_level` and `dbg_pid` are read on every `dprintf`. They are only written when someone runs `dbg`. I did not use a lock.

Why it is correct: each variable is aligned and fits in a register. On RV64 a read is one `ld` or `lw`, and a write is one `sd` or `sw`. A reader gets the old value or the new value, never half of each. All reads and writes use `__atomic_load_n` and `__atomic_store_n` with `__ATOMIC_RELAXED`. This makes each access atomic and stops the compiler from keeping an old value in a register. It doesn't add any fences. The only race is that a message printed right when the setting changes might or might not show up. That doesn't matter for logging. `dbg_test` reads `dbg_pid` once, so it compares against one value.

Why not a lock: the check runs on hot paths on every CPU. A lock would make all the CPUs wait on a value that almost never changes. `printf` also takes `pr.lock`, so another lock here would add a lock order to worry about for no reason.

Limitations: the SET ops read the old value and then write the new one. These are two separate steps. So two SET calls at the same time could both return the same old value. `__atomic_exchange_n` would fix this. Also, `dbg on` and `dbg off` read the mask, change it, and write it back from user space. Two `dbg` commands at the exact same time could lose one change. `dbg` is run by hand, so I accepted both. Kernel ops that set or clear bits with `__atomic_fetch_or` and `__atomic_fetch_and` would fix the second one.

7. Bonus points

a. Did you submit the Bonus “Start Strong” survey by the deadline? Yes.

b. Did you attempt the extra functionality for bonus points? Yes. Filter by pid.

i. Description. There are two new `debugctl` ops, `DBGCTL_GETPID` (4) and `DBGCTL_SETPID` (5). They control a new variable, `dbg_pid`. 0 means no filter. `SETPID` rejects values that don't fit in an int. `dbg_test` checks the pid last, after the mask and level. So messages that are already off don't cost anything extra. The `dbg` tool has `dbg pid <n>` and `dbg pid off`, and plain `dbg` shows the current filter. The filter matches the process that is running when the message prints. It does not match pids named in the message. For example, “parent 2 reaped child 5” runs in pid 2. It shows with `dbg pid 2` but not with `dbg pid 5`.

ii. Console output. `transcripts/7_bonus_pid_filter.txt`. With `dbg pid 2`, `echo hi` only shows the shell's `kfork` and `kwait`. The child's `kexec` and `kexit` are hidden. After `dbg pid off`, all four show again. `dbg pid abc` and `dbg pid` with nothing after it are rejected.